

1. WORKING OF CNN ARCHITECTURE TO CLASSIFY IMAGES

EX.N0 : 1	WORKING OF CNN ARCHITECTURE TO CLASSIFY IMAGES
<u>DATE : 21/01/2025</u>	

AIM:

To implement and demonstrate a Convolutional Neural Network (CNN) for image classification using the MNIST dataset in TensorFlow and Keras.

ALGORITHM:

Step 1: Import necessary libraries.

Step 2: Load and pre-process the MNIST dataset.

Step 3: Build the CNN model using Keras Sequential API.

Step 4: Compile the model with appropriate loss function and optimizer.

Step 5: Train the model on the training dataset.

Step 6: Evaluate the model on the test dataset.

Step 7: Visualize a few predictions to validate performance.

PROGRAM:

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist
import matplotlib.pyplot as plt
import numpy as np

# Load MNIST dataset for simplicity
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
train_images = train_images.reshape((60000, 28, 28, 1)).astype('float32') / 255.0
test_images = test_images.reshape((10000, 28, 28, 1)).astype('float32') / 255.0
```

```

# Create a simple CNN model
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])

# Compile the model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Train the model
history = model.fit(train_images, train_labels, epochs=5, validation_data=(test_images,
test_labels))

# Evaluate the model
test_loss, test_acc = model.evaluate(test_images, test_labels)

print(f"Test Accuracy: {test_acc}")

# Plotting some images along with model predictions
def plot_images(images, labels, predictions=None, num_images=5):
    fig, axes = plt.subplots(1, num_images, figsize=(12, 3))
    for i in range(num_images):
        img = images[i].reshape(28, 28)
        ax = axes[i]
        ax.imshow(img, cmap='gray')
        if predictions is not None:
            ax.set_title(f"Label: {labels[i]} \nPred: {predictions[i]}")

```

```

else:
    ax.set_title(f"Label: {labels[i]}")
    ax.axis('off')
plt.show()

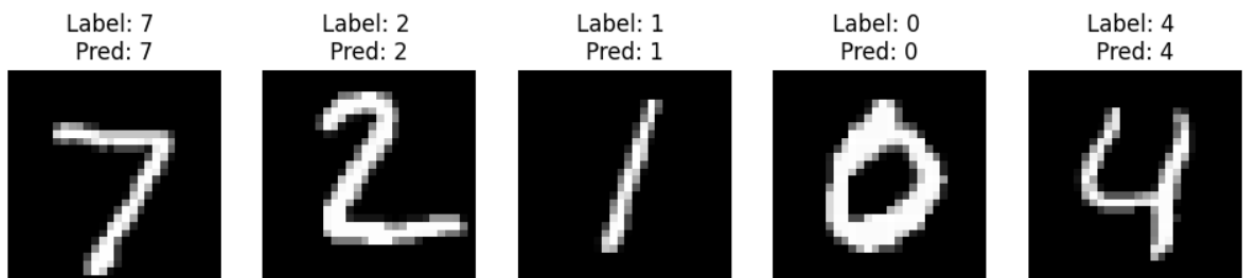
# Get predictions for test images
predictions = np.argmax(model.predict(test_images[:5]), axis=-1)

# Display some images with their predicted labels
plot_images(test_images, test_labels, predictions)

```

OUTPUT:

313/313 ——— 35 yms/step - accuracy: 0.9864 - loss: 0.0394
Test Accuracy: 0.9897000193595886
1/1 ——— 0s 137ms/step



RESULT:

Thus, to implement and demonstrate a Convolutional Neural Network (CNN) for image classification using the CIFAR-10 dataset in TensorFlow and Keras.